

University of Pittsburgh

Introduction to Operating Systems
CS 1550

Spring 2022

Sherif Khattab

ksm73@pitt.edu

(Some slides are from **Silberschatz, Galvin and Gagne ©2013)**

Announcements

- Upcoming deadlines:
  - Lab 3: due on 4/1
  - Homework 9: due on 4/4
  - Project 3: due on 4/11

Previous lecture …

- Large Page Table problem
  - multi-level page tables
  - inverted page tables

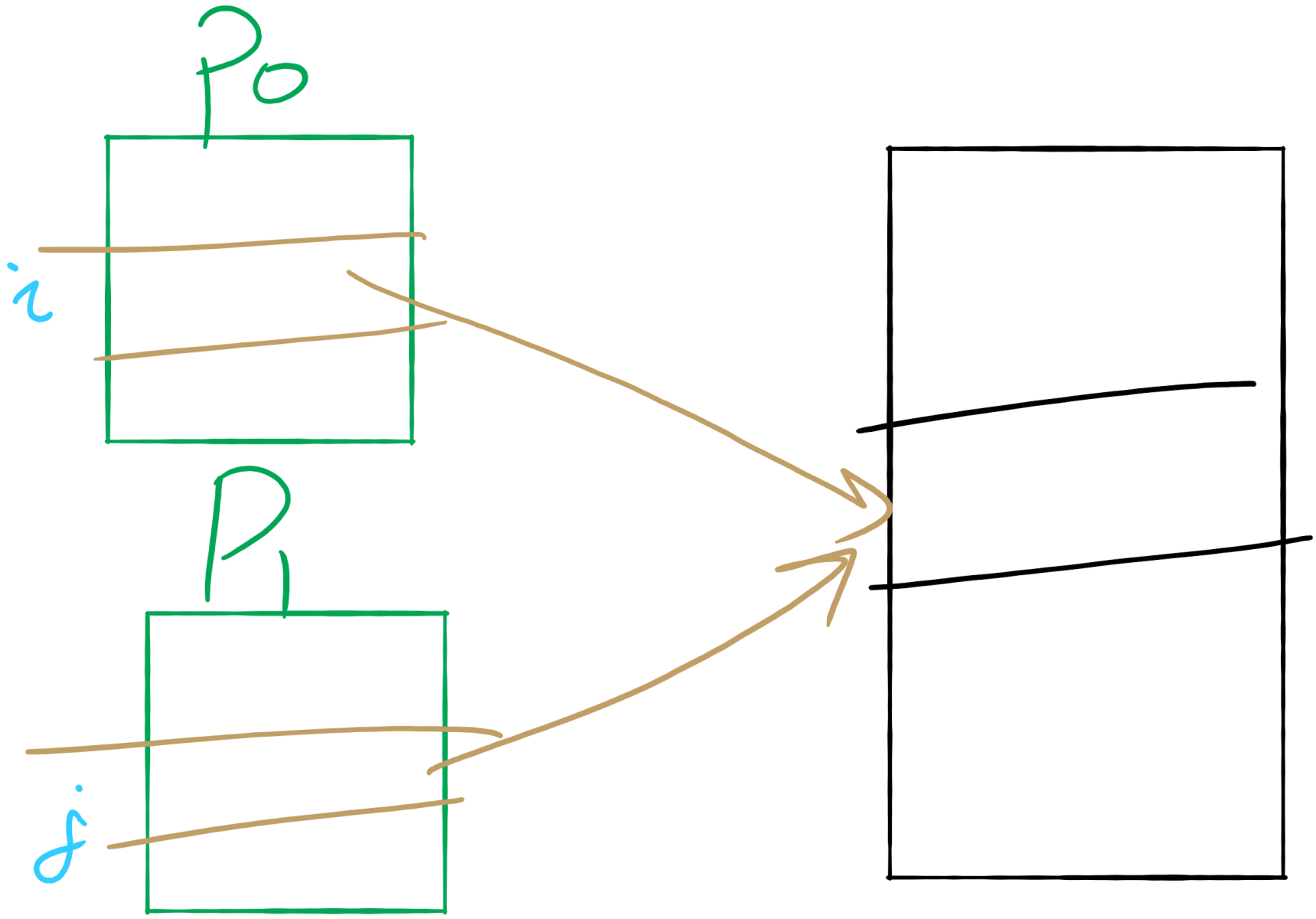
Muddiest Points

- EAT
- Inverted page tables
- Would you please solve a problem about Inertial Page Table? Thank you so much
- why again will there be multiple memory accesses using a TLB?
- Two level page tables and how they are better than single page tables (take up less space)
- Where are the second and third level page tables stored? Are they paged in and out of memory like a normal page?
- when will we get our midterm grades

How big should a page be?

- Smaller pages have advantages
  - Less internal fragmentation
  - Better fit for various data structures, code sections
  - Less unused physical memory (some pages have 20 useful bytes and the rest isn't needed currently)
- Larger pages are better because
  - Less overhead to keep track of them
    - Smaller page tables
    - TLB can point to more memory (same number of pages, but more memory per page)
    - Faster paging algorithms (fewer table entries to look through)
  - More efficient to transfer larger pages to and from disk

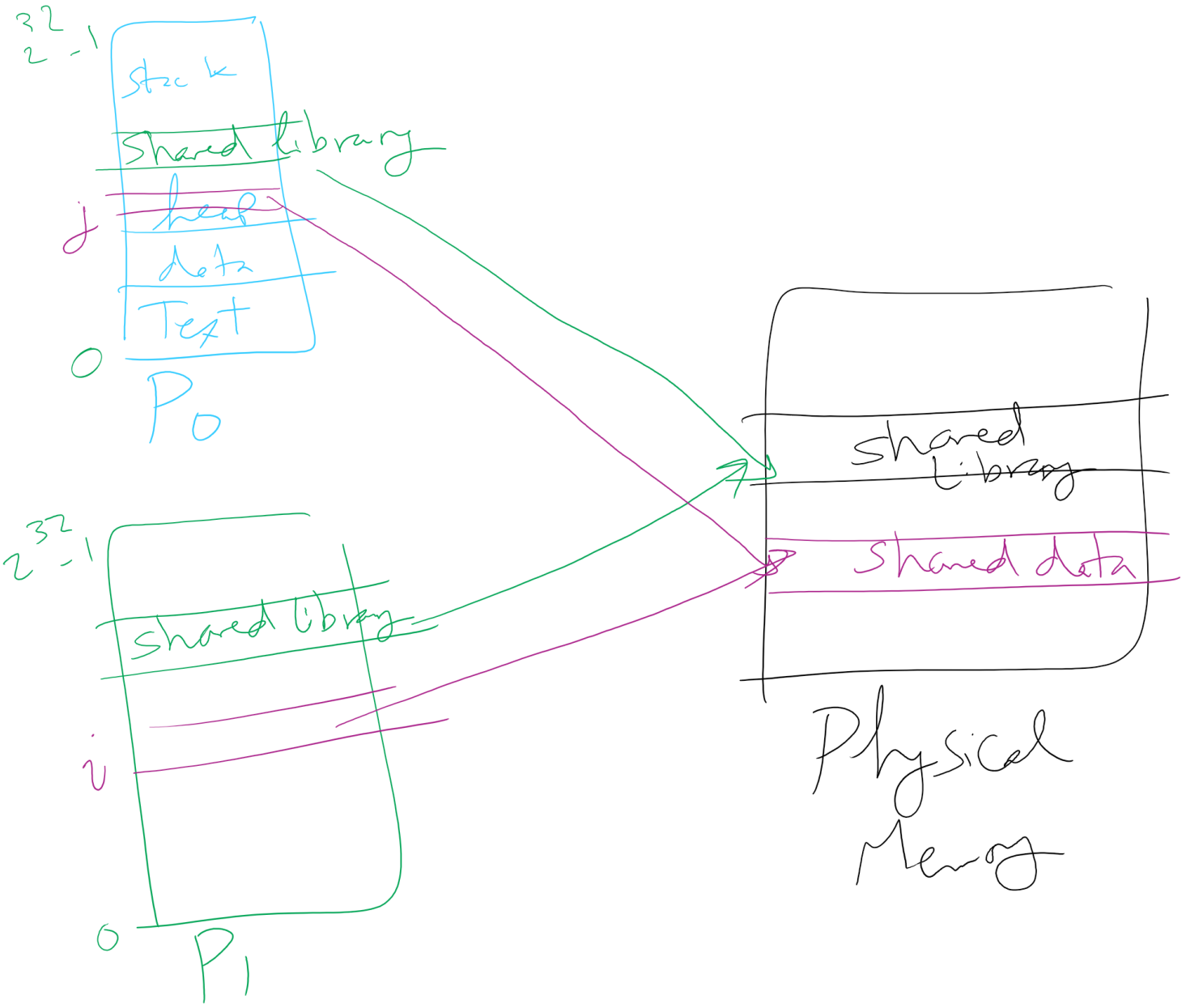
Sharing Pages



Sharing pages

- Processes can share pages
  - Entries in page tables point to the same physical page frame
  - Easier to do with code: no problems with modification
- Virtual addresses in different processes can be…
  - The same: easier to exchange pointers, keep data structures consistent
  - Different: may be easier to actually implement
    - Not a problem if there are only a few shared regions
    - Can be very difficult if many processes share regions with each other

Page Sharing



Implementation issues

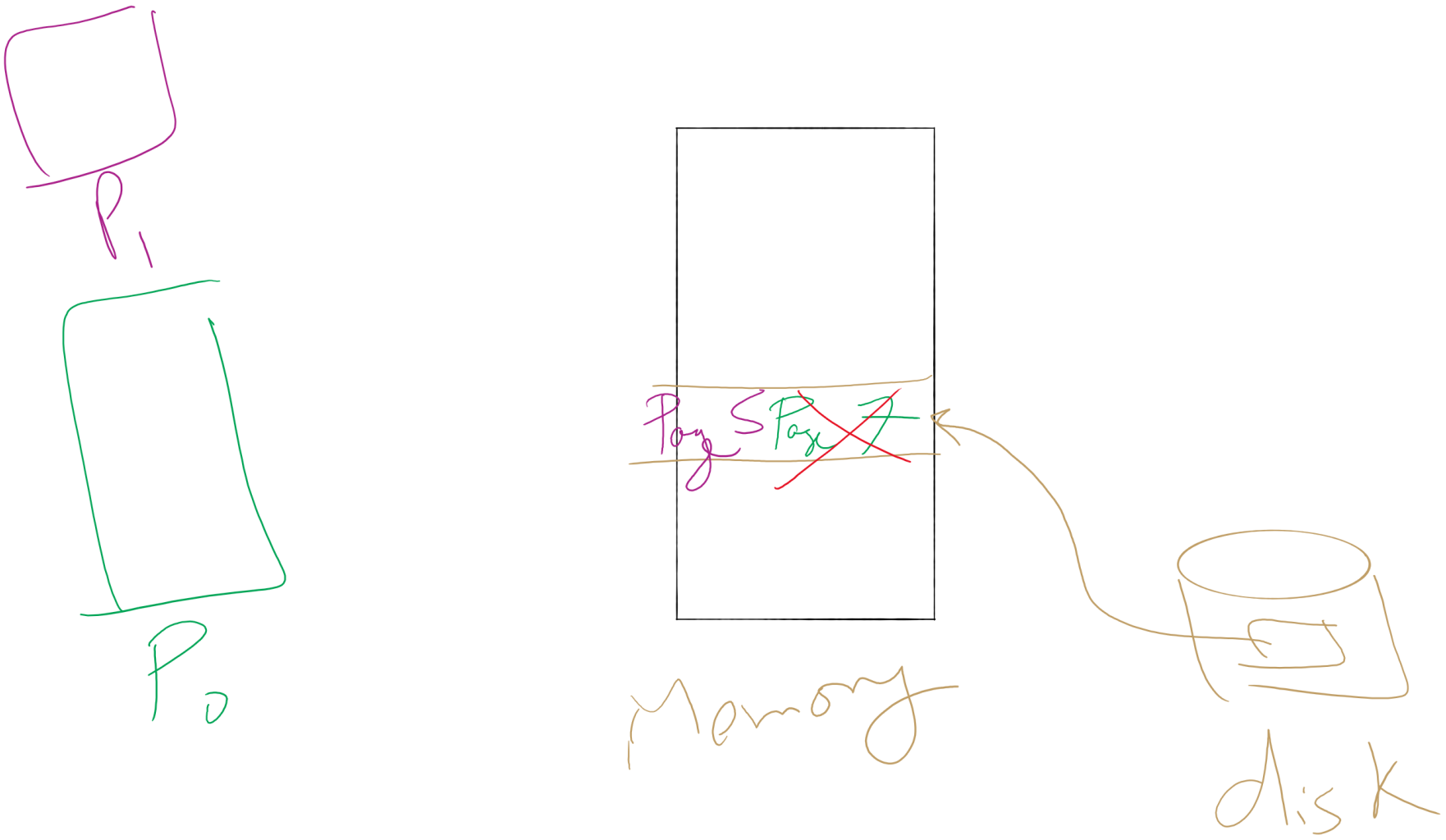
Four times when OS involved with paging

- Process creation
  - Determine program size
  - Create page table
- During process execution
  - Reset the MMU for new process
  - Flush the TLB (or reload it from saved state)
- Page fault time
  - Determine virtual address causing fault
  - Swap target page out, needed page in
- Process termination time
  - Release page table
  - Return pages to the free pool

How is a page fault handled?

- Hardware causes a page fault
- General registers saved (as on every exception)
- OS determines which virtual page needed
  - Actual fault address in a special register
  - Address of faulting instruction in register
    - Page fault was in fetching instruction, or
    - Page fault was in fetching operands for instruction
    - OS must figure out which…
- OS checks validity of address
  - Process killed if address was illegal
- OS finds a place to put new page frame
- If frame selected for replacement is dirty, write it out to disk
- OS requests the new page from disk
- Page tables updated
- Faulting instruction backed up so it can be restarted
- Faulting process scheduled
- Registers restored
- Program continues

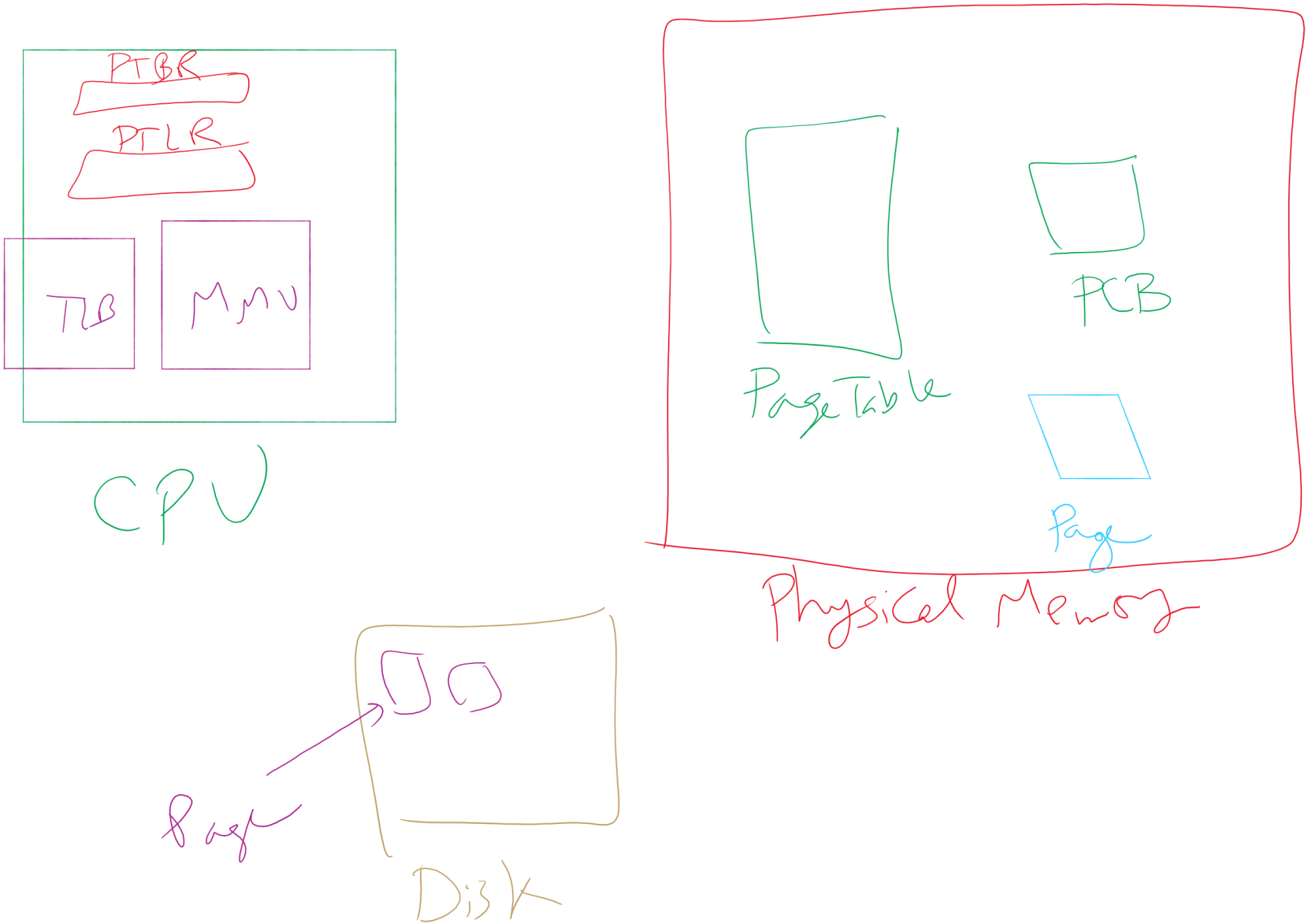
Page locking



Locking pages in memory

- Virtual memory and I/O occasionally interact
- P1 issues call for read from device into buffer
  - While it's waiting for I/O, P2 runs
  - P2 has a page fault
  - P1's I/O buffer might be chosen to be paged out
    - This can create a problem because an I/O device is going to write to the buffer on P1's behalf
- Solution: allow some pages to be *locked* into memory
  - Locked pages are immune from being replaced
  - Pages only stay locked for (relatively) short periods

Map of MMU, TLB, Page Table, etc.



When are dirty pages written to disk?

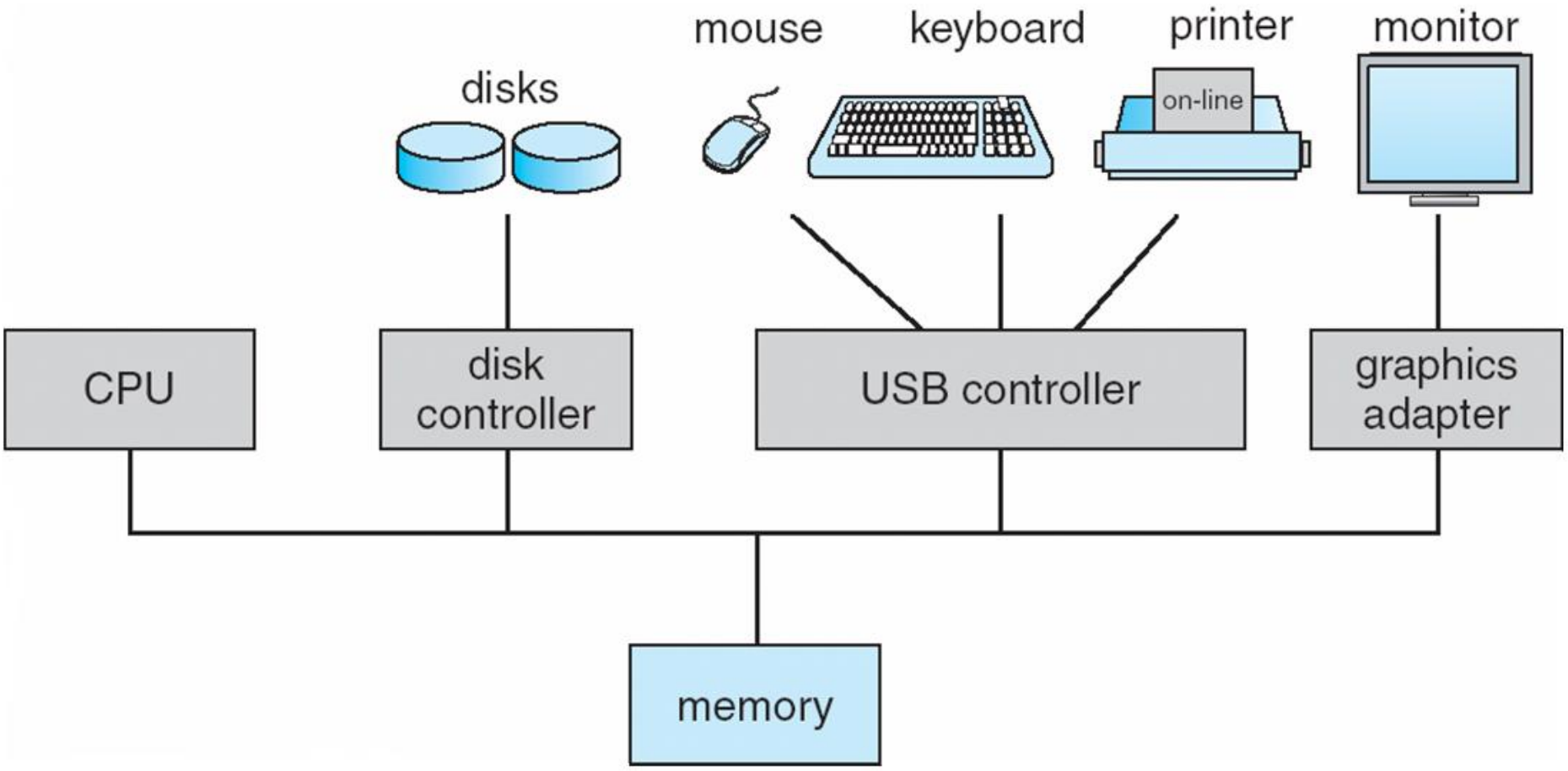
- On demand (when they're replaced)
  - Fewest writes to disk
  - Slower: replacement takes twice as long (must wait for disk write ***and*** disk read)
- Periodically (in the background)
  - Background process scans through page tables, writes out dirty pages that are pretty old
- Background process also keeps a list of pages ready for replacement
  - Page faults handled faster: no need to find space on demand
  - Cleaner may use the same structures discussed earlier (clock, etc.)

Problem of the Day

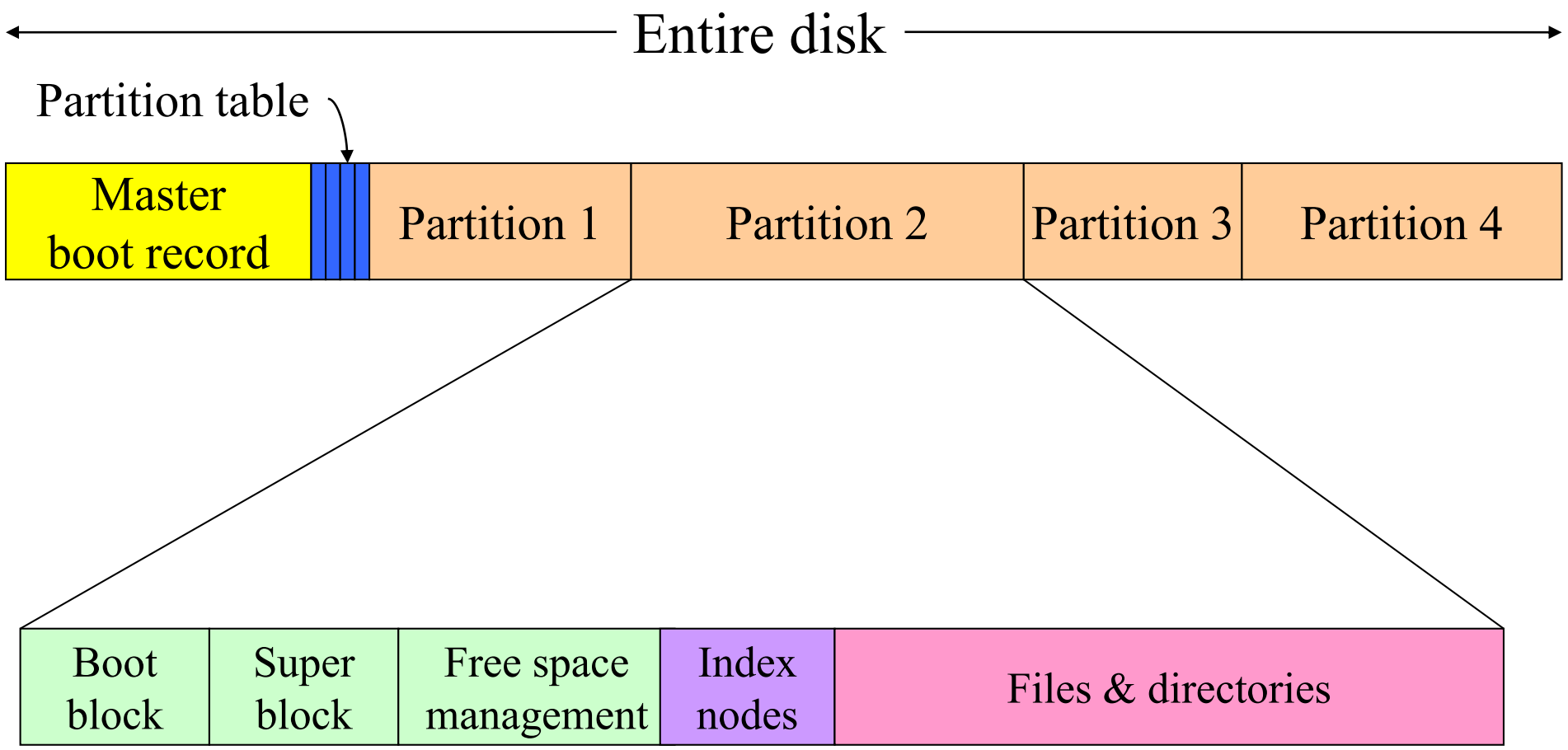
How to allocate disk blocks to files and directories?

How complex is the OS's job?

Let's look at one of the resources managed by the OS: I/O devices

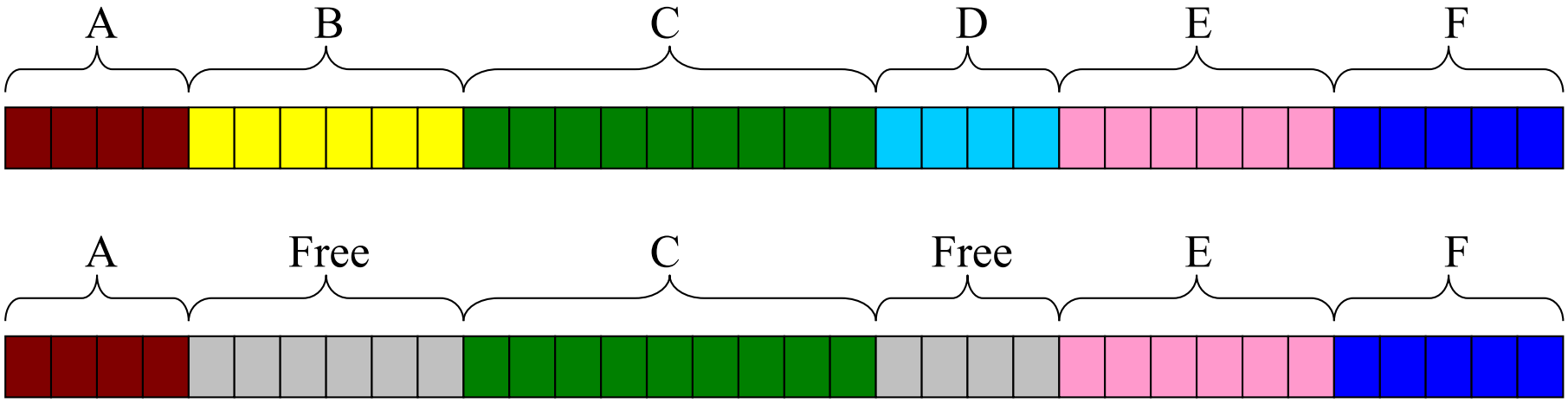


Carving up the disk



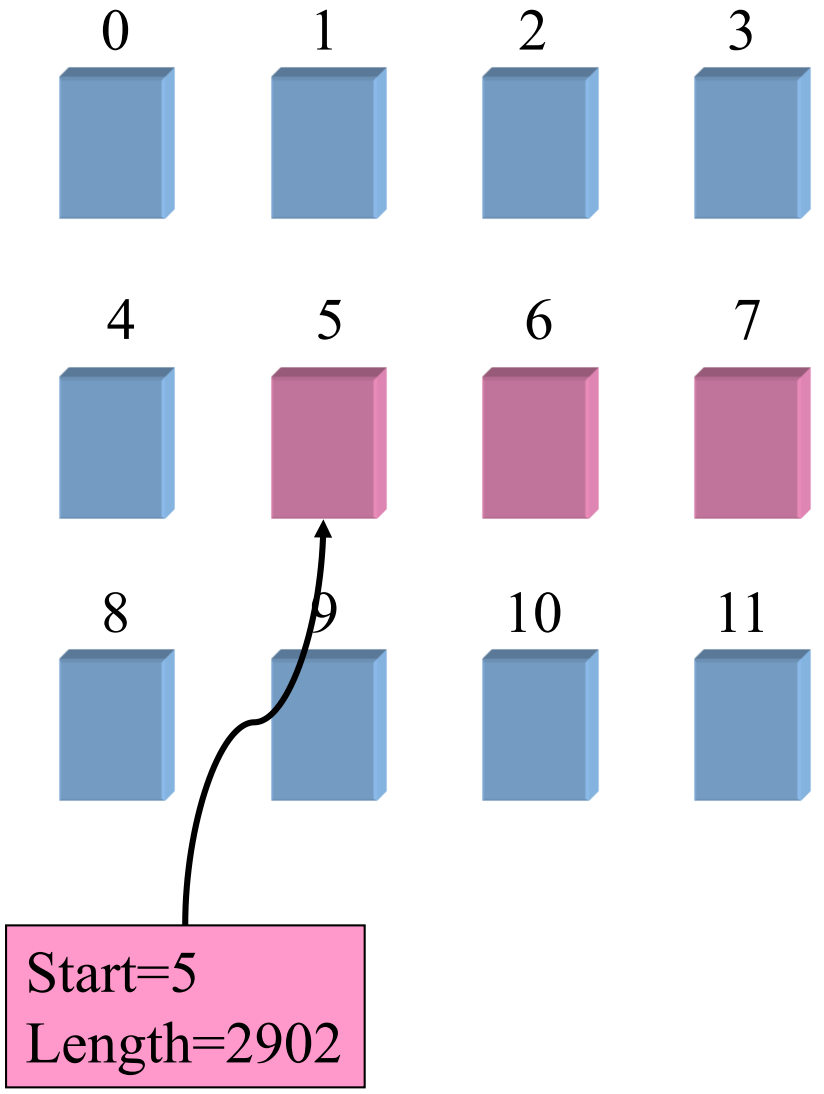
Contiguous allocation for file blocks

- Contiguous allocation requires all blocks of a file to be consecutive on disk
- Problem: deleting files leaves “holes”
  - Similar to memory allocation issues
  - Compacting the disk can be a very slow procedure…



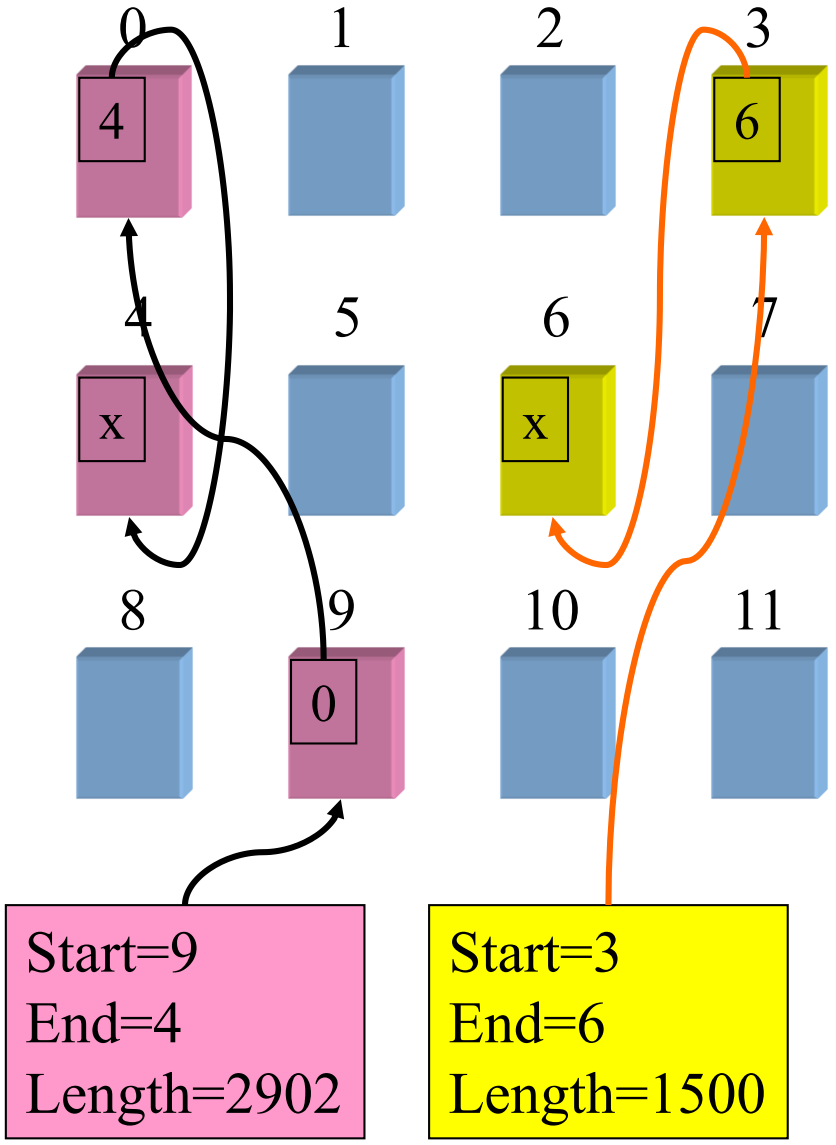
Contiguous allocation

- Data in each file is stored in consecutive blocks on disk
- Simple & efficient indexing
  - Starting location (block #) on disk (start)
  - Length of the file in blocks (length)
- Random access well-supported
- Difficult to grow files
  - Must pre-allocate all needed space
  - Wasteful of storage if file isn't using all of the space
- Logical to physical mapping is easy
  blocknum = (pos / 1024) + start;
  offset_in_block = pos % 1024;



Linked allocation

- File is a linked list of disk blocks
  - Blocks may be scattered around the disk drive
  - Block contains both pointer to next block and data
  - Files may be as long as needed
- New blocks are allocated as needed
  - Linked into list of blocks in file
  - Removed from list (bitmap) of free blocks



Data Structures for Linked Allocation

```
struct disk_block {
    unsigned int next_block;
    unsigned char data[1020];
}
```

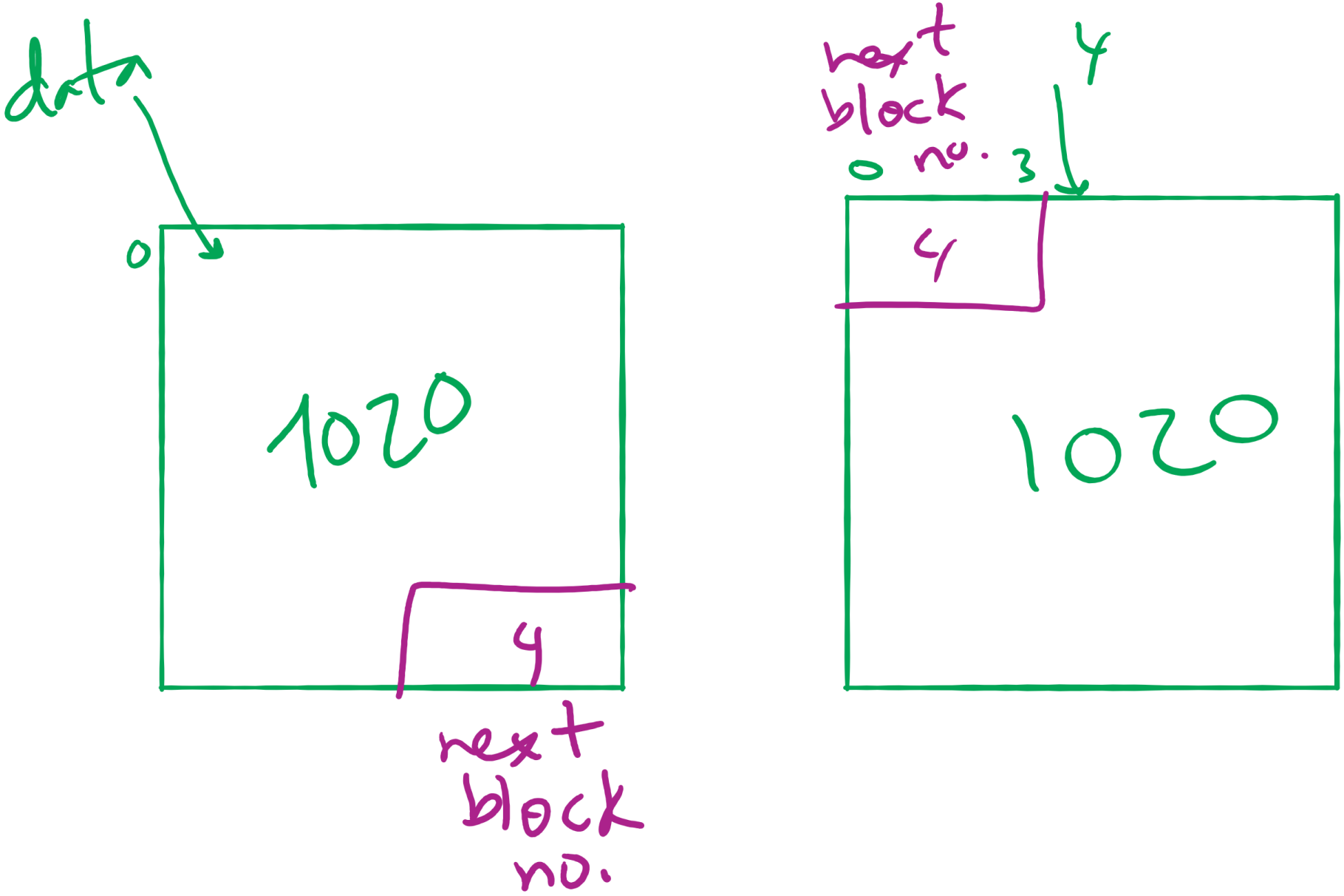
Finding blocks with linked allocation

- Directory structure is simple
  - Starting address looked up from directory
  - Directory only keeps track of first block (not others)
- No wasted blocks - all blocks can be used
- Random access is difficult: must always start at first block!
- Logical to physical mapping is done by

```
block = start;
offset_in_block = pos % 1020;
for (j = 0; j < pos / 1020; j++) {
  block = block->next;
}
```

  - Assumes that next pointer is stored at end of block
  - May require a long time for seek to random location in file

Offset Calculation



File Allocation Table (FAT)

- Links on disk are slow
- Keep linked list in memory
- Advantage: faster
- Disadvantages
  - Have to copy it to disk at some point
  - Have to keep in-memory and on-disk copy consistent

